

# Mutliple Parrallel Solutions to A\* Search

Omar Attaalla

June 2025

## 1 Introduction

Many of today's optimization problems include some sort of searching algorithm. In particular, today, most autonomous vehicles and robots make use of the popular path finding algorithm A\*. A\* (and optimal search in general) is computationally intensive for uninformed graphs with both running and space complexity that is exponential in the length of the shortest path from a given node to a goal node. Therefore, A\* would benefit drastically from being parallelized across multiple cores. However, A\* is not naturally parallelizable since it is built on breadth first search which is not capable of naturally dividing work across cores. Instead, this paper discusses and implements two solutions to make A\* parallelizable: Simple Parallel A\* (SPA\*) and Decentralized Parallel A\*. We will discuss the challenges that come with both solutions and compare the performance between these two parallel A\* solutions and a standard single-core A\* implementation.

A\* uses a heuristic to guide the search. The performance varies greatly based on how well the heuristic function estimates the cost to the goal. In our implementation of A\*, we use a priority queue to expand nodes with the minimal cost (heuristic + cost from starting node).

## 2 Standard A\* Implementation

The standard A\* implementation is simple with a single priority queue that is sorted by  $f(n)$ , the cost from the start node to the current node and the heuristic. The algorithm expands the starting node then loops (expanding the next highest priority node) until no nodes exist in the priority queue.

To simulate A\*, we create a Grid class that constructs widthxheight nodes that contain an x,y coordinate, a g cost, a heuristic cost, and if the node is an obstacle. The class Node is shown below:

```
class Node {
    public:
        int x;
        int y;
        int g; // Cost from start node
        int h; // Heuristic cost to goal node
        bool isObstacle; // Flag to indicate if the node is an obstacle
        bool closedSet = false;

        Node(int x, int y, bool isObs, int g = 0, int h = 0) :
            x(x), y(y), g(g), h(h), isObstacle(isObs) {}

        int getF() const {
            return g + h; // Total cost
        }
};
```

In this implementation, we use the simple manhattan distance heuristic:

```
startNode->h = abs(startNode->x - goalNode->x) + abs(startNode->y - goalNode->y);
```

However, we also included a loop to simulate a heavy workload when computing the manhattan distance (since in real applications of A\*, computing the heuristic is computationally intensive). This impacts the performance, since multi-threading will lead to a larger improvement if the expansion of a node takes long. Otherwise, the context switch and locking overhead will lead the standard A\* to have better performance in every case.

```
int temp = 0;
for (int i = 0; i < 10000; ++i) {
    temp += i * i;
}
volatile int sink = temp;
```

To simplify things, we set the start node as Grid[0][0] and the goal as Grid[width - 1][height - 1]. The A\* algorithm then follows by utilizing the priority queue to expand the "best" nodes first. We implement the standard A\* algorithm to benchmark and compare the performance to the parallel versions.

### 3 Simple Parallel A\*

Simple Parallel A\* (SPA\*), also known as Centralized Parallel A\*, works by controlling access to a shared priority queue that stores the open set [1]. This control is handled by using locks. In particular, we benchmarked A\* when using different lock types: Spin Lock, MCS Lock, CLH Lock. This implementation is very similar to the standard A\* implementation. The only difference is the use of locks to avoid multiple concurrent access to a shared priority queue. The full implementation is included with this report, but we would like to highlight where the locks are used. To extract the most promising node from the priority queue, we need to lock the priority queue to ensure the other threads are not accessing it at the same time:

```
lock->lock();
if (openSet.empty()) {
    lock->unlock();
    return;
}
current = openSet.top();
openSet.pop();
lock->unlock();
```

And with updating/pushing neighbors, we also need to lock the shared queue. Note that when we update the heuristic, we still use the "simulated heavy work" that was discussed earlier:

```
neighbor->g = currG;
neighbor->h = abs(neighbor->x - goalNode->x) + abs(neighbor->y - goalNode->y);
lock->lock();
openSet.push(neighbor);
lock->unlock();
```

### 4 Decentralized Parallel A\*

The decentralized parallel A\* algorithm attempts to avoid having a shared open set since this is where most of the overhead and lock contention occurs in the Centralized Parallel A\* algorithm. Additionally, each thread maintains its own lock. When nodes are added to the open set, a thread is assigned as the owner

of that node using a multiplicative hash function using the (x,y) coordinate. Each thread also maintains its own open set. When a node is expanded, the node is pushed onto the owner of that (x,y) coordinate's priority queue. We use the struct ThreadInfo to denote the additional info maintained by the thread (open set, lock):

```
struct ThreadInfo {
    std::priority_queue<Node*, std::vector<Node*>, Compare> openSet;
    Lock* lock;
};
```

The full implementation of the Decentralized Parallel A\* is included with this report. It includes the use of ThreadInfo as well as the multiplicative hash which evenly distributes ownership of nodes (based on x,y coordinates) across all the threads. If a thread owns a node, any thread must push that node to the owner's priority queue for later expansion.

## 5 Evaluation

### 5.1 Setup

To evaluate the performance of the Decentralized and Centralized A\* algorithms, we implemented a test bench that was run on a 16 core AMD 16 core processor. We varied the grid size of each A\* problem from 250x250 to 1500x1500 (all grids were square). We initialized each grid with a random set of obstacles (we ensured that a solution exists in each problem and that each algorithm ran on the same grid with the same obstacles to ensure fairness in the tests. We create a dynamic testAStar method:

```
std::chrono::duration<double> testAStar(Grid mainGrid, int numThreads,
                                         LockType lockType, bool centralized) {
    Grid grid(mainGrid);

    if (lockType == LockType::NONE) { //standard A*
        auto start = std::chrono::high_resolution_clock::now();
        AStar(grid);
        auto end = std::chrono::high_resolution_clock::now();
        std::chrono::duration<double> elapsed = end - start;
        return elapsed;
    }

    if (centralized) {
        Lock* lock = LockFactory::createLock(lockType);

        if (lock) {
            std::chrono::duration<double> elapsed =
                parallelAStarCentralized(lock, numThreads, grid);
            delete lock;
            return elapsed;
        }
    } else {
        std::chrono::duration<double> elapsed =
            parallelAStarDecentralized(grid, numThreads, lockType);
        return elapsed;
    }
}
```

```

    return std::chrono::duration<double>(0);
}

```

We make use of the `std::chrono::high resolution clock` to ensure that the performance measurements of of high resolution. Additionally, we vary the number of threads in each iteration (from 1 - 16 threads) to see how performance scales with a different number of threads.

The `testAStar` method takes as input a grid, the number of threads, a lock type and if we are using the centralized algorithm. This made the implementation simpler since we benefit from much code reuse. Based on these inputs, the correct test was run and timed. The results were output to a csv file.

## 5.2 Results

### 5.2.1 Centralized vs Decentralized A\*

The results from running the Parallel Centralized and Decentralized A\* algorithm are shown below. We also include a standard A\* run that runs on a single thread to compare the performance improvements.

| Grid Size | Threads | Centralized(S) | Decentralized(S) | Standard  |
|-----------|---------|----------------|------------------|-----------|
| 250x250   | 1       | 0.0167802      | 0.0175582        | 0.0219191 |
| 250x250   | 2       | 0.0035134      | 0.004924         | 0.0210579 |
| 250x250   | 4       | 0.0016515      | 0.003441         | 0.0214123 |
| 250x250   | 8       | 0.0027914      | 0.0025264        | 0.0209628 |
| 250x250   | 16      | 0.0033701      | 0.0041561        | 0.0212741 |
| 500x500   | 1       | 0.0632943      | 0.0658707        | 0.0797909 |
| 500x500   | 2       | 0.0274693      | 0.0335658        | 0.0803605 |
| 500x500   | 4       | 0.0095109      | 0.0163887        | 0.080367  |
| 500x500   | 8       | 0.0055483      | 0.0081696        | 0.0820296 |
| 500x500   | 16      | 0.011674       | 0.0114403        | 0.0815496 |
| 750x750   | 1       | 0.167761       | 0.164763         | 0.2025    |
| 750x750   | 2       | 0.0577866      | 0.0731895        | 0.206482  |
| 750x750   | 4       | 0.0130576      | 0.0277887        | 0.210649  |
| 750x750   | 8       | 0.009306       | 0.0112925        | 0.200056  |
| 750x750   | 16      | 0.0222545      | 0.0138203        | 0.205207  |
| 1000x1000 | 1       | 0.309418       | 0.312945         | 0.368473  |
| 1000x1000 | 2       | 0.0788898      | 0.134884         | 0.381623  |
| 1000x1000 | 4       | 0.0267944      | 0.0536537        | 0.366013  |
| 1000x1000 | 8       | 0.0163179      | 0.017516         | 0.380818  |
| 1000x1000 | 16      | 0.0222678      | 0.0212143        | 0.379271  |
| 1250x1250 | 1       | 0.393255       | 0.398054         | 0.498253  |
| 1250x1250 | 2       | 0.101627       | 0.188135         | 0.494339  |
| 1250x1250 | 4       | 0.0226183      | 0.0588328        | 0.516068  |
| 1250x1250 | 8       | 0.0186612      | 0.0206946        | 0.499866  |
| 1250x1250 | 16      | 0.0429898      | 0.0228678        | 0.508142  |
| 1500x1500 | 1       | 0.756589       | 0.765277         | 0.924539  |
| 1500x1500 | 2       | 0.185723       | 0.366333         | 0.908987  |
| 1500x1500 | 4       | 0.0538906      | 0.0685664        | 0.951785  |
| 1500x1500 | 8       | 0.0256406      | 0.0419411        | 0.903499  |
| 1500x1500 | 16      | 0.0597067      | 0.0256892        | 0.912844  |

Table 1: Centralized vs Decentralized vs Standard A\* w/ Spin Lock

We can see in table 1 above that the parallel algorithms performed much better than the standard single

thread A\*. We also see that overall the Decentralized algorithm performed better and scaled better when the grid size was increased. For example, on the 1500x1500, we can see that the Decentralized Parallel A\* algorithm ran over 2 times faster than the Centralized Parallel A\* algorithm. And to compare with the standard A\* algorithm, the Decentralized Parallel A\* algorithm ran over 35 times faster than the single-threaded standard A\* implementation.

### 5.2.2 Locking Mechanisms

We also chose to measure the performance differences between different locking mechanism's. In particular, we utilized MCS, CLH, and Spin Locks. The performance results are shown below (with 16 threads and varying grid size).

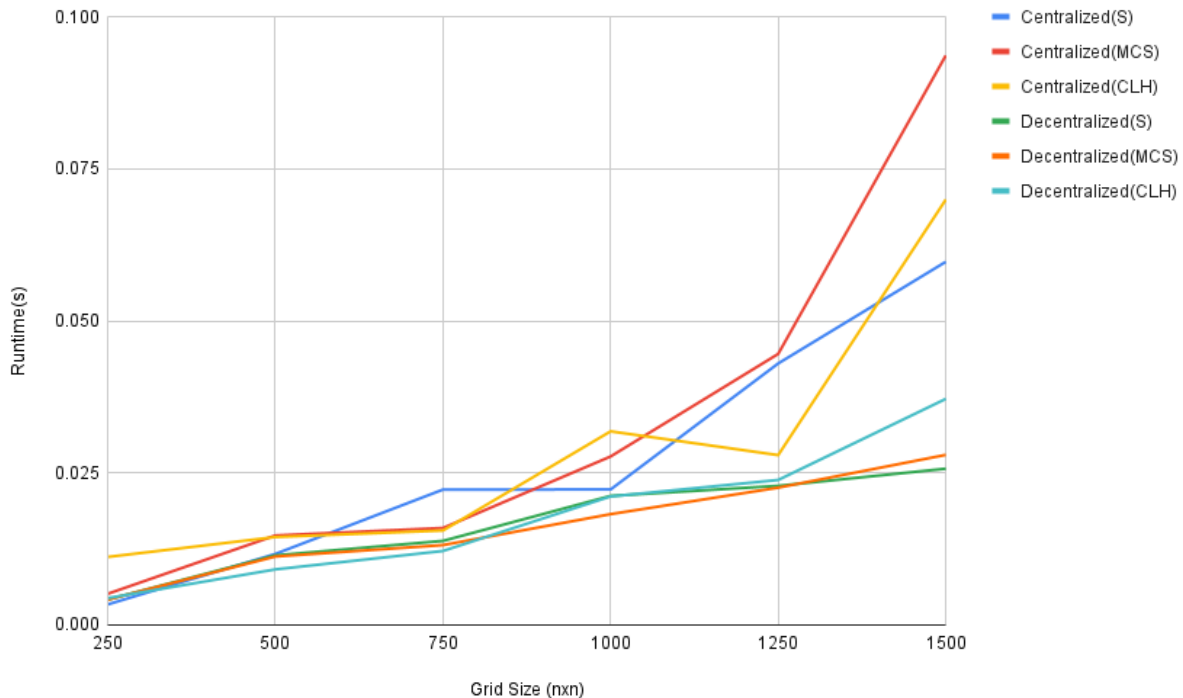


Figure 1: Centralized/Decentralized A\* Runtime with Varying Locks

Interestingly, all locks performed similarly. However, when using the Centralized algorithm, CLH locks generally performed slightly better. On the other hand, the standard spin lock performed best when using the Decentralized algorithm. One reason for this is since CLH locks lower cache contention and spin on local memory, it performed well when using the shared priority queue. However, when each thread maintains its own local priority queue, cache coherence is not as important (less memory is shared between threads) and the standard spin lock performed best (least overhead). We can also see that the Decentralized algorithm scales much better as the grid size is increased. We see an almost linear trend of the runtime of the Decentralized algorithm, compared to an exponential one when using the Centralized algorithm.

## 6 Conclusion

In this paper, we implemented both a Centralized Parallel A\* algorithm and Decentralized Parallel A\* algorithm. They differ in the way they handle ownership of the open set. The centralized algorithm maintains a shared priority queue, allowing a single thread to push or pop from it at a time. This led to some performance bottlenecks as more threads attempted to acquire the lock to the shared thread. On the other hand, the Decentralized Parallel A\* algorithm maintained a lock and priority queue that is local to each thread. This made contention less likely to occur since there are an equal amount of threads as the number of open sets.

We then gauged the performance of these two algorithms and compared it with a standard A\* implementation. As expected, the Decentralized Parallel A\* algorithm performed the best, while the Centralized Parallel A\* algorithm was still a significant improvement over the single threaded A\* algorithm. The Decentralized performed about 2x better than the Centralized algorithm and over 35 times better than the standard A\* implementation.

Finally, we also compared how different locks (MCS, Spin, CLH) performed with the different algorithms. We found that the Centralized algorithm performed best with the CLH lock since it improved contention by making lock work local. On the other hand, the simple spin lock performed best when using the decentralized algorithm since the reduction in contention did not matter as much (because most work was local to each thread). The reduced overhead of the using the spin lock yielded the best results in the decentralized algorithm.

## References

- [1] Alex Fukunaga, Adi Botea, Yuu Jinnai, and Akihiro Kishimoto. A survey of parallel a\*. 08 2017.